
大数据与人工智能
自动化学院

CHAPTER3-1： PYTORCH基础--PYTORCH的数据载体张量与线性回归

3.1-01 导言

深度学习、大语言模型、大数据的很多操作都需要借助深度学习代码框架。

深度学习框架

- 是一种软件库或工具，可以帮助开发者构建和训练深度神经网络模型。
- 深度神经网络是一种人工智能算法，受到人类大脑结构和功能的启发，可以从大量的数据中学习复杂的特征和规律。
- 深度学习框架提供了高层次的接口，库和工具，简化了复杂的机器学习模型的开发过程。

目前，有很多不同的深度学习框架，各有其优缺点和特色。

- 一些比较流行和广泛使用的深度学习框架有：
    - TensorFlow：由谷歌开发的开源深度学习框架，支持分布式计算，多种平台和设备，以及多种机器学习任务。TensorFlow使用张量（多维数组）来表示神经网络中的数据和操作，因此得名。
    - Keras：一个基于TensorFlow和Theano的高层次深度学习框架，提供了简洁和直观的API，方便初学者和快速原型设计。
    - PyTorch：由Facebook开发的开源深度学习框架，强调动态计算图和Python风格的编程，使得研究者和开发者可以更容易地实验和迭代模型。

- Caffe：由UC Berkeley开发的开源深度学习框架，特别适合计算机视觉领域的应用，如图像识别和检测。Caffe具有非常出色的卷积神经网络实现，但缺乏对循环神经网络等其他类型的深度学习应用的支持。
- Theano：深度学习框架中的元老，使用Python编写。Theano派生出了大量Python深度学习库，最著名的包括Keras。Theano非常灵活，适合做学术研究的实验，但速度较慢。
- Deeplearning4j：一个基于Java和Scala编写的开源深度学习框架，支持分布式计算和Hadoop集成。Deeplearning4j适合大规模数据处理和商业应用。

PYTORCH是目前运用最广泛的深度学习框架之一

主要有以下几个原因：

- PyTorch支持动态计算图，即在运行时可以根据输入数据动态地改变神经网络结构。这使得PyTorch更加灵活和自然地表达复杂的逻辑和控制流程。
- PyTorch遵循Python语言风格和惯例，使得代码更加简洁和易读。PyTorch还与Python科学计算库如NumPy, SciPy等兼容良好，方便数据处理和分析。
- PyTorch拥有丰富的文档和社区资源，为开发者提供了大量的教程，示例，论坛等。PyTorch还受到许多知名机构如Facebook, Uber, Stanford等的支持和贡献。
- PyTorch在研究领域非常受欢迎，因为它可以快速地实现新颖的想法和算法，并且易于调试。
- PyTorch还提供了一些高级功能如自动微分（autograd），分布式训练（distributed training）等。

3.1-01 导言

我们直接从深度学习框架中的PYTORCH开始

- 运用非常广泛
- 代码比较容易编写
- 比较容易从算法到代码进行转化和理解

本节内容

- 首先整理Pytorch中数据结构张量的概念和创建方法
- 然后整理张量的各种操作
- 最后通过前面所学玩一个简单的线性回归。

关于PYTORCH的安装

课后自行安装CUDA

搭建PYTORCH环境

- 如果你电脑是Windows，并且没有装Pytorch，那么可以先看看（Windows下的Pytorch适合比较简单的深度学习开发）
  https://blog.csdn.net/wuzhongqiang/article/details/104503860
- Linux下安装Pytorch，比较适合进阶的深度学习、大数据的研究人员（因为有很多代码编译、库之间的依赖关系，在Linux系统里面资源比较丰富，有的直接现成就能用，比如GCC编译器）
- Ubuntu下的深度学习框架安装-2022.docx

3.1 PYTORCH的数据结构

3.1-2. 张量的简介与创建

这部分内容介绍PYTORCH中的数据结构--TENSOR

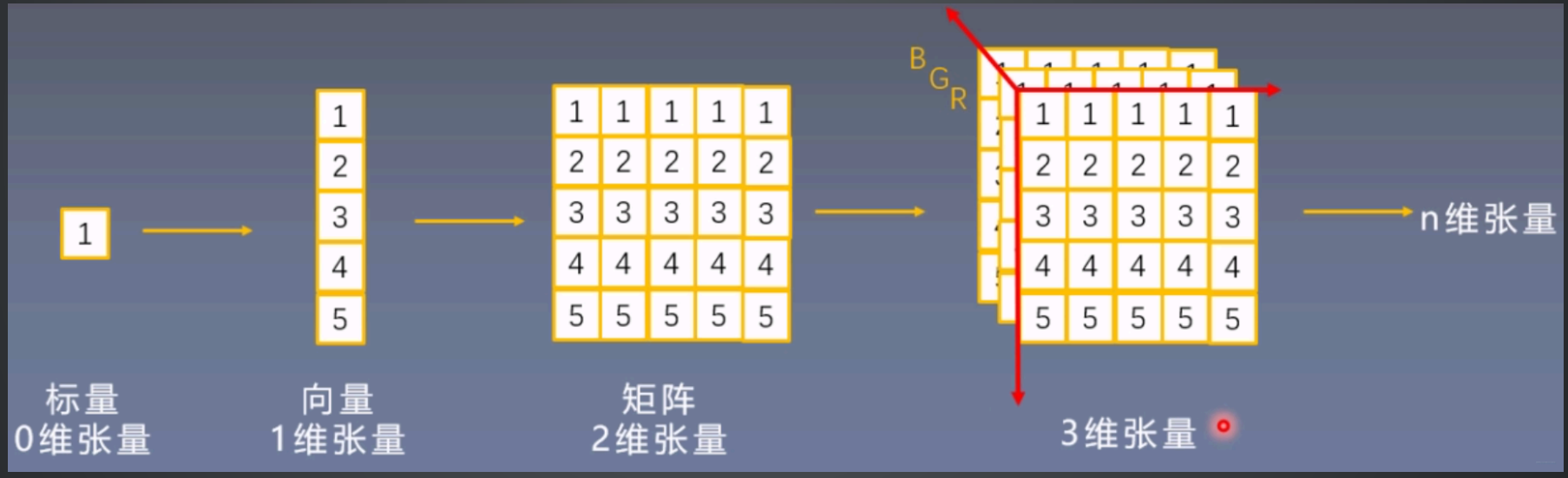
张量(TENSOR)是PYTORCH中最基础的概念，其参与了整个运算过程

主要介绍

- 张量的概念和属性，如data, device, dtype等
- 介绍tensor的基本创建方法，如直接创建、依数值创建和依概率分布创建等。

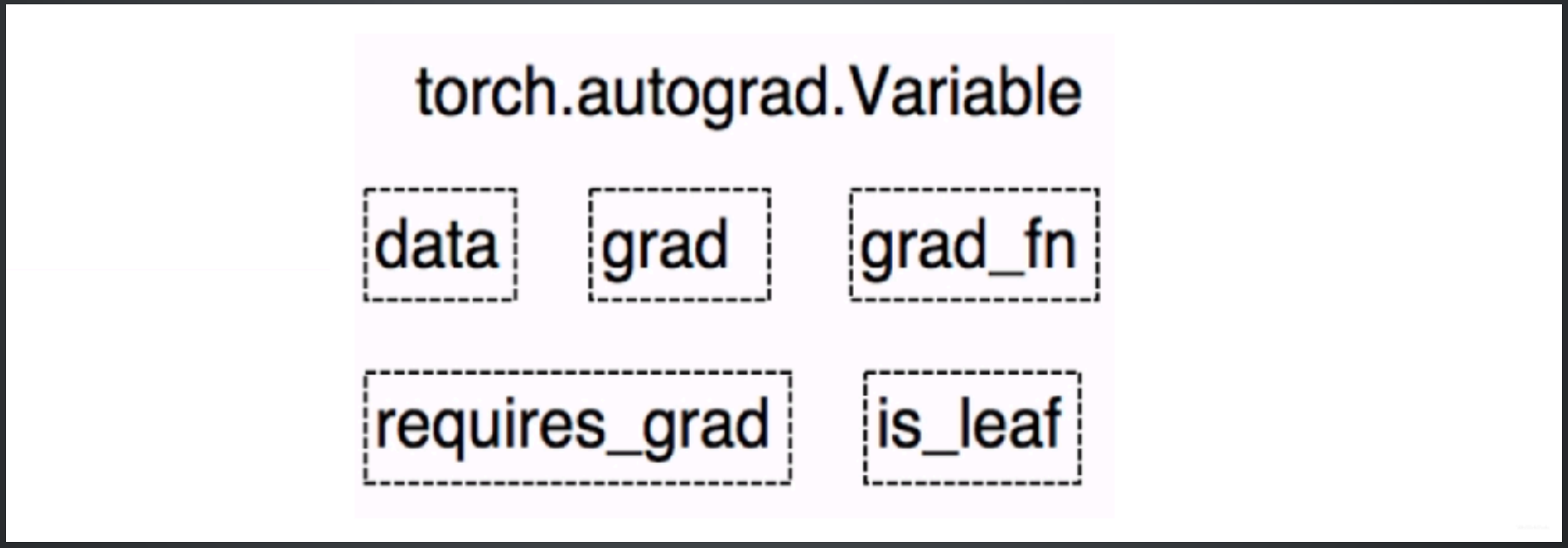
3.1-2.1 张量的简介

1. 张量的基本概念
   张量其实是一个**多维数组**，它是标量、向量、矩阵的高维拓展



2. Tensor与Variable
在Pytorch0.4.0版本之后其实Variable已经并入Tensor， 但是Variable这个数据类型的了解，对于理解张量来说很有帮助

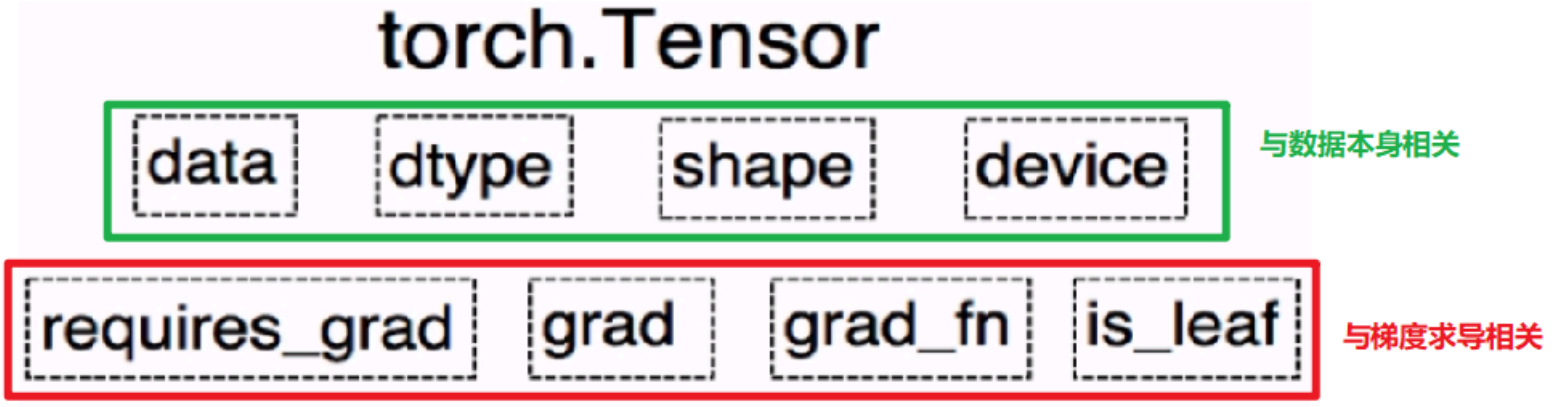
- Variable到底是个什么呢?
Variable是torch.autograd中的数据类型。



Variable有下面的5个属性：

- *data: 被包装的Tensor*
- *grad: data的梯度*
- *grad_fn: fn表示function的意思，记录我创建的创建张量时用到的方法，比如说加法，乘法，这个操作在求导过程需要用到，Tensor的Function，是自动求导的关键*
- *requires_grad: 指示是否需要梯度，有的不需要梯度*
- *is_leaf: 指示是否是叶子节点（张量）*

这些属性都是为了张量的自动求导而设置的， 从Pytorch0.4.0版开始，Variable并入了Tensor， 看看张量里面的属性：



可以发现，如今版本里面的Tensor共有8个属性

- 上面四个与数据本身相关
- 下面四个与梯度求导相关。
- 其中有五个是Variable并入过来的， 这些含义就不解释了

而还有三个属性没有说：

- *dtype: 张量的数据类型， 如torch.FloatTensor, torch.cuda.FloatTensor, 用的最多的一般是float32和int64(torch.long)*
- *shape: 张量的形状， 如(64, 3, 224, 224)*
- *device: 张量所在的设备， GPU/CPU， 张量放在GPU上才能使用加速。*

知道了什么是张量，那么如何创建张量呢?

3.1-2.2 张量的创建

1. 直接创建张量

torch.Tensor(): 功能： 从data创建Tensor

```
torch.tensor(
            data,
            dtype=None,
            device=None,
            requires_grad=False,
            pin_memory=False)
```

- 这里的data，就是我们的数据，可以是list，也可以是numpy。
- dtype这个是指明数据类型， 默认与data的一致。
- device是指明所在的设备
- requires_grad是是否需要梯度， 在搭建神经网络的时候需要求导的那些参数这里要设置为true。
- pin_memory是否存于锁页内存，这个设置为False就可以。

下面就具体代码演示：

```
arr = np.ones((3, 3))
print('ndarry的数据类型：', arr.dtype)

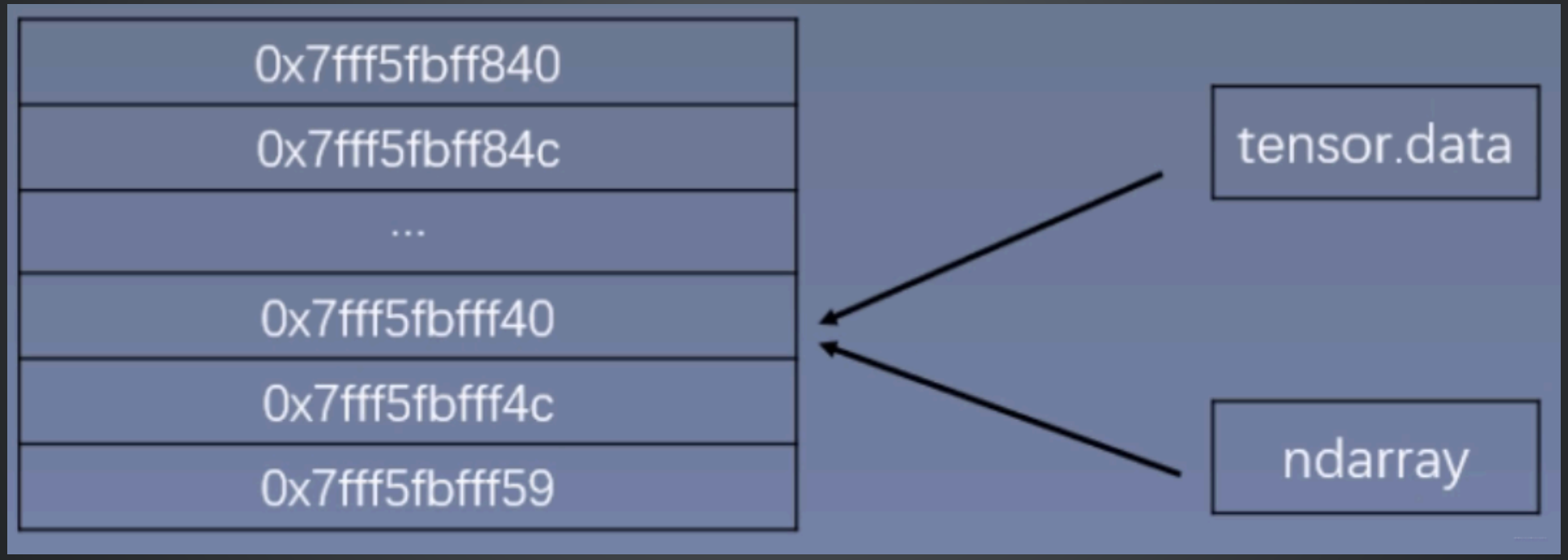
t = torch.tensor(arr, device='cuda')
print(t)

## 结果如下：
ndarry的数据类型： float64
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]], device='cuda:0', dtype=torch.float6
```

2. 通过NUMPY数组来创建

torch.from_numpy(ndarry): 从numpy创建tensor

注意：这个创建的Tensor与原ndarray**共享内存**, 当修改其中一个数据的时候，另一个也会被改动。



```
arr = np.array([[1, 2, 3], [4, 5, 6]])
t = torch.from_numpy(arr)

print(arr, '\n',t)
arr[0, 0] = 0
print('*' * 10)
print(arr, '\n',t)
t[1, 1] = 100
print('*' * 10)
print(arr, '\n',t)
```

```
## 结果:
[[1 2 3]
 [4 5 6]]
 tensor([[1, 2, 3],
         [4, 5, 6]], dtype=torch.int32)
**********
[[0 2 3]
 [4 5 6]]
 tensor([[0, 2, 3],
         [4, 5, 6]], dtype=torch.int32)
**********
```

```
[[  0   2   3]
 [  4 100   6]]
```

3. 依据数值创建

torch.zeros(): 依size创建全0的张量

```
torch.zeros(*size,
            out=None,
            dtype=None,
            layout=torch.strided,
            device=None,
            requires_grad=False)
```

- 这些参数都比较好理解
- layout这个是内存中的布局形式， 一般采用默认就可以。
- out，表示输出张量，就是再把这个张量赋值给别的一个张量，但是这两个张量是一样的，指的同一个内存地址。看代码：

```
out_t = torch.tensor([1])
t = torch.zeros((3, 3), out=out_t)

print(out_t, '\n', t)
print(id(t), id(out_t), id(t) == id(out_t))    # 这个看内

## 结果:
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
 tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

torch.zeros_like(input, dtype=None, layout=None, device=None, requires_grad=False) : 这个是创建与input同形状的全0张量

```
t = torch.zeros_like(out_t)    # 这里的input要是个张量
print(t)

tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

除了全0张量， 还可以创建全1张量， 用法和上面一样：

- torch.ones()
- torch.ones_like()

还可以自定义数值张量：torch.full(), torch.full_like()

```
torch.full(size,
           fill_value,
           out=None,
           dtype=None,
           layout=torch.strided,
           device=None,
           requires_grad=False)
```

这里的fill_value就是要填充的值。

```
t = torch.full((3,3), 10)
tensor([[10., 10., 10.],
        [10., 10., 10.],
        [10., 10., 10.]])
```

提问：以下哪个函数可以用来创建一个全1的张量？

- A) torch.zeros()
- B) torch.ones()
- C) torch.rand()
- D) torch.randn()

正确答案： B) torch.ones()

torch.ones()函数用于创建一个全1的张量。类似地，torch.zeros()用于创建全0的张量，而torch.rand()和torch.randn()分别用于生成均匀分布和标准正态分布的随机数张量

torch.arange(): 创建等差的1维张量，数值区间[start, end)， **注意这是右边开，取不到最后的那个数。**

```
torch.arange(start=0,
             end,
             step=1,
             out=None,
             dtype=None,
             layout=torch.strided,
             device=None,
             requires_grad=False)
```

这个和numpy的差不多，这里的step表示的步长。

```
t = torch.arange(2, 10, 2)   # tensor([2, 4, 6, 8])
```

torch.linspace(): 创建均分的1维张量， 数值区间[start, end] 注意这里都是闭区间，和上面的区分。

```
torch.linspace(start,
               end,
               steps=100,
               out=None,
               dtype=None,
               layout=torch.strided,
               device=None,
               requires_grad=False)
```

这里是右闭， 能取到最后的值，并且这里的steps是数列的长度而不是步长（但是这个是间隔的数目）。

```
t = torch.linspace(2, 10, 5)    # tensor([2, 4, 6, 8, 1

# 那么如果不是那么正好呢？ 步长应该是多少?
t = torch.linspace(2, 10, 6)    # tensor([2, 3.6, 5.2,

# 这个步长是怎么算的?   (end-start) / (steps-1)
```

除了创建均分数列，还可以创建对数均分数列：

```
torch.logspace(start,
               end,
               steps=100,
               base=10.0,
               out=None,
               dtype=None,
               layout=torch.strided,
               device=None,
               requires_grad=False)
```

这里的base表示以什么为底

最后一个方法就是**torch.eye(): 创建单位对角矩阵， 默认是方阵**

```
torch.eye(n,
          m=None,
          out=None,
          dtype=None,
          layout=torch.strided,
          device=None,
          requires_grad=False)
```

n, m分别是矩阵的行数和列数。

```
t = torch.eye(3,4)
print(t)
```

4. 依概率分布创建张量

torch.normal():生成正态分布（高斯分布）， 这个使用的比较多

```
torch.normal(mean,
             std,
             size,
             out=None)
```

mean是均值，std是标准差。 但是这个地方要注意， 根据mean和std，分别各有两种取值，所以这里会有四种模式：

- mean为标量， std为标量
- mean为标量， std为张量
- mean为张量， std为标量
- mean为张量，std为张量

这个看代码来的直接：

```
 # 第一种模式 - 均值是标量， 方差是标量 - 此时产生的是一个分布，
t_normal = torch.normal(0, 1, size=(4,))
print(t_normal)      # 来自同一个分布

# 第二种模式 - 均值是标量， 方差是张量 - 此时会根据方差的形状大
std = torch.arange(1, 5, dtype=torch.float)
print(std.dtype)
t_normal2 = torch.normal(1, std)
print(t_normal2)          # 也产生来四个数，但是这四个数分别来

# 第三种模式 - 均值是张量，方差是标量 - 此时也会根据均值的形状大
mean = torch.arange(1, 5, dtype=torch.float)
t_normal3 = torch.normal(mean, 1)
```

下面一个是标准正态分布：均值为0，方差为1 torch.randn(), torch.randn_like()

```
torch.randn(*size,
            out=None,
            dtype=None,
            layout=torch.strided,
            device=None,
            requires_grad=False)
```

生成均匀分布：torch.rand(), rand_like() 在[0,1)生成均匀分布
生成整数均匀分布：torch.randint(), torch.randint_like(): 区间[low,hight)生成整数均匀分布

```
torch.rand(*size,
           out=None,
           dtype=None,
           layout=torch.strided,
           device=None,
           requires_grad=False)
torch.randint(low=0,
              high,
              size,
              out=None,
              dtype=None,
              layout=torch.strided,
              device=None,
              requires_grad=False)
```

下面看最后两个：

torch.randperm(n): 生成从0 ~ n-1的随机排列, n是张量的长度, 经常用来生成一个乱序索引。

torch.bernoulli(input): 以input为概率，生成伯努利分布(0-1分布，两点分布)， input: 概率值

```
torch.randperm(n,
               out=None,
               dtype=torch.int64,
               layout=torch.strided,
               device=None,
               requires_grad=False)

torch.bernoulli(input,
                *,
                generator=None,
                out=None)
```

3.1-3. 张量的操作

下面整理张量的基本操作

- 比如张量的拼接，切分，索引和变换以及数学运算等
- 基于所学习的知识，实现线性回归模型。

3.1-3.1 张量的基本操作

1. 张量的拼接

- **torch.cat(tensors, dim=0, out=None): 将张量按维度dim进行拼接, tensors表示张量序列， dim要拼接的维度**
- **torch.stack(tensors, dim=0, out=None): 在新创建的维度dim上进行拼接， tensors表示张量序列， dim要拼接的维度**

- cat直接在原来维度上面拼接
- 看代码：

```
# 张量的拼接
t = torch.ones((2, 3))
print(t)

t_0 = torch.cat([t, t], dim=0)        # 行拼接
t_1 = torch.cat([t, t], dim=1)     # 列拼接
print(t_0, t_0.shape)
print(t_1, t_1.shape)

# 结果:
tensor([[1., 1., 1.],
    [1., 1., 1.]])
tensor([[1., 1., 1.],
```

- stack会新创建一个维度，然后完成拼接。
- 下面我们看看stack方法：

```
t_stack = torch.stack([t,t,t], dim=0)
print(t_stack)
print(t_stack.shape)

t_stack1 = torch.stack([t, t, t], dim=1)
print(t_stack1)
print(t_stack1.shape)

## 结果:
tensor([[[1., 1., 1.],
         [1., 1., 1.]],

        [[1., 1., 1.],
```

- stack是根据给定的维度新增了一个新的维度，在这个新维度上进行拼接
- 这个stack与其说是从新维度上拼接，不太好理解，其实是新加了一个维度Z轴，只不过dim=0和dim=1的视角不同罢了。 dim=0的时候，是横向看，dim=1是纵向看。

所以cat和stack这两个使用的时候要小心，看好了究竟是在原来的维度上拼接到一块，还是从新维度上拼接到一块。

提问：在PyTorch中，torch.cat()和torch.stack()的主要区别是什么？

- A) torch.cat()拼接新维度，torch.stack()在已有维度上拼接
- B) torch.cat()在已有维度上拼接，torch.stack()拼接新维度
- C) 两者功能相同，只是名字不同
- D) torch.cat()用于切分张量，torch.stack()用于拼接张量

正确答案： B) torch.cat()在已有维度上拼接，torch.stack()拼接新维度

- 讲评： torch.cat()是在已有维度上拼接多个张量 而torch.stack()会新增一个维度再进行拼接。例如，如果你有两个形状为(2, 3)的张量，用cat(dim=0)会得到形状为(4, 3)，而用stack(dim=0)则会得到形状为(2, 2, 3)1

2. 张量的切分

torch.chunk(input, chunks, dim=0): 将张量按维度dim进行平均切分， 返回值是张量列表，注意，如果不能整除， 最后一份张量小于其他张量。 chunks代表要切分的维度。*

- 下面看一下代码实现：

```
a = torch.ones((2, 7))  # 7
list_of_tensors = torch.chunk(a, dim=1, chunks=3)    # 1
print(list_of_tensors)
for idx, t in enumerate(list_of_tensors):
        print("第{}个张量：{}, shape is {}".format(idx+1
```

```
## 结果:
(tensor([[1., 1., 1.],
         [1., 1., 1.]]), tensor([[1., 1., 1.],
         [1., 1., 1.]]), tensor([[1.],
         [1.]]))
第1个张量: tensor([[1., 1., 1.],
         [1., 1., 1.]]), shape is torch.Size([2, 3]
第2个张量: tensor([[1., 1., 1.],
         [1., 1., 1.]]), shape is torch.Size([2, 3]
第3个张量: tensor([[1.],
         [1.]]), shape is torch.Size([2, 1])
```

**torch.split(tensor, split_size_or_sections, dim=0):

- 这个也是将张量按维度dim切分，但是这个更加强大， 可以指定切分的长度，
- split_size_or_sections为int时表示每一份的长度， 为list时，按list元素切分

```
# split
t = torch.ones((2, 5))

list_of_tensors = torch.split(t, [2, 1, 2], dim=1)
for idx, t in enumerate(list_of_tensors):
    print("第{}个张量: {}, shape is {}".format(idx+1
```

```
## 结果
第1个张量: tensor([[1., 1.],
        [1., 1.]]), shape is torch.Size([2, 2])
第2个张量: tensor([[1.],
        [1.]]), shape is torch.Size([2, 1])
第3个张量: tensor([[1., 1.],
        [1., 1.]]), shape is torch.Size([2, 2])
```

总结：切分，有两个函数，.CHUNK和.SPLIT

- .chunk切分的规则就是提供张量，切分的维度和几份， 比如三份， 先计算每一份的大小，也就是
  这个维度的长度除以三，然后上取整，就开始沿着这个维度切，最后不够一份大小的，也就那样了。 所以长度为7的这个维度，3块，每块7/3上取整是3， 然后第一块3，第二块是3，第三块1。这样切
- .split这个函数的功能更加强大，它可以指定每一份的长度，只要传入一个列表即可，或者也有一个整数，表示每一份的长度，这个就根据每一份的长度先切着， 看看能切几块算几块。
    - 列表的那个好使，可以自己指定每一块的长度，但是注意一下，这个长度的总和必须是维度的那个总长度才用这个办法切。

3. 张量的索引

- torch.index_select(input, dim, index, out=None):
- 在维度dim上，按index索引数据，返回值，以index索引数据拼接的张量。

```
t = torch.randint(0, 9, size=(3, 3))      # 从0-8随机产生
print(t)
idx = torch.tensor([0, 2], dtype=torch.long)    # 这里的
t_select = torch.index_select(t, dim=1, index=idx)  #
print(t_select)
```

```
## 结果:
tensor([[3, 7, 3],
        [4, 3, 7],
        [5, 8, 0]])
tensor([[3, 3],
        [4, 7],
        [5, 0]])
```

torch.masked_select(input, mask, out=None):

- 按mask中的True进行索引
- 返回值：一维张量。
- input表示要索引的张量，
- mask表示与input同形状的布尔类型的张量。 这种情况在选择符合某些特定条件的元素的时候非常好使， 注意这个是返回一维的张量。下面看代码：

```
mask = t.ge(5)    # le表示<=5, ge表示>=5 gt >5  lt <5
print("mask: \n", mask)
t_select1 = torch.masked_select(t, mask)    # 选出t中大于
print(t_select1)
```

```
## 结果:
mask:
 tensor([[False,  True, False],
    [False, False,  True],
    [ True,  True, False]])
tensor([7, 7, 5, 8])
```

小结：张量的索引，有两种方式：.index_select和.masked_select

- index_select： 按照索引查找 需要先指定一个Tensor的索引量，然后指定类型是long的
- masked_select: 就是按照值的条件进行查找，需要先指定条件作为mask

4. 张量的变换

torch.reshape(input, shape): 变换张量的形状

- 这个很常用. input表示要变换的张量，shape表示新张量的形状。
- 注意，当张量在内存中是连续时， 新张量与input共享数据内存

```
# torch.reshape
t = torch.randperm(8)          # randperm是随机排列的一个函数
print(t)

t_reshape = torch.reshape(t, (-1, 2, 2))     # -1的话就是
print("t:{}\nt_reshape:\n{}".format(t, t_reshape))

t[0] = 1024
print("t:{}\nt_reshape:\n{}".format(t, t_reshape))
print("t.data 内存地址:{}".format(id(t.data)))
print("t_reshape.data 内存地址:{}".format(id(t_reshape.d
```

```
## 结果:
tensor([2, 4, 3, 1, 5, 6, 7, 0])
t:tensor([2, 4, 3, 1, 5, 6, 7, 0])
t_reshape:
tensor([[[2, 4],
         [3, 1]],

        [[5, 6],
         [7, 0]]])
t:tensor([1024,    4,    3,    1,    5,    6,    7,
t_reshape:
tensor([[[1024,    4],
         [   3,    1]],
```

上面这两个是共内存的，一个改变另一个也会改变。这个要注意一下。

torch.transpose(input, dim0, dim1): 交换张量的两个维度

- 矩阵的转置常用， 在图像的预处理中常用
- dim0要交换的第1个维度， dim1表示要交换的第2个维度

```
# torch.transpose
t = torch.rand((2, 3, 4))        # 产生0-1之间的随机数
print(t)
t_transpose = torch.transpose(t, dim0=1, dim1=2)     # 
print("t shape:{}\nt_transpose shape: {}".format(t.sha
```

```
## 结果:
tensor([[[0.7738, 0.4975, 0.1435, 0.2425],
         [0.1790, 0.7871, 0.5170, 0.8049],
         [0.7568, 0.4127, 0.6050, 0.5078]],
        [[0.9680, 0.6753, 0.8548, 0.8645],
         [0.9530, 0.2427, 0.3309, 0.3039],
         [0.4342, 0.7475, 0.8399, 0.2517]]])
t shape:torch.Size([2, 3, 4])
t_transpose shape: torch.Size([2, 4, 3])
```

torch.t(input): 2维张量的转置， 对矩阵而言，相当于torch.transpose(input, 0,1)

torch.squeeze(input, dim=None, out=None): 压缩长度为1的维度， dim若为None，移除所有长度为1的轴，若指定维度，当且仅当该轴长度为1时可以被移除

```
# torch.squeeze
t = torch.rand((1, 2, 3, 1))
t_sq = torch.squeeze(t)
t_0 = torch.squeeze(t, dim=0)
t_1 = torch.squeeze(t, dim=1)
print(t.shape)          # torch.Size([1, 2, 3, 1])
print(t_sq.shape)       # torch.Size([2, 3])
print(t_0.shape)      # torch.Size([2, 3, 1])
print(t_1.shape)      # torch.Size([1, 2, 3, 1])
```

torch.unsqueeze(input, dim, out=None): 依据dim扩展维度

3.1-3.2 张量的数学运算

3.1-3.2.1 标量运算

1. Pytorch中提供了丰富的数学运算，可以分为三大类： 加减乘除， 对数指数幂函数，三角函数 。注意：这些都是对应位置的标量计算

```
torch.add()
torch.addcdiv()
torch.addcmul()
torch.sub()
torch.div()
torch.mul()
```

张量的数学运算

```
torch.log(input, out=None)
torch.log10(input, out=None)
torch.log2(input, out=None)
torch.exp(input, out=None)
torch.pow()

torch.abs(input, out=None)
torch.acos(input, out=None)
torch.cosh(input, out=None)
torch.cos(input, out=None)
torch.asin(input, out=None)
torch.atan(input, out=None)
torch.atan2(input, other, out=None)
```

2. 这里重点演示一下加法这个函数， 因为这个函数有一个小细节：

torch.add(input, alpha=1, other, out=None)

- 逐元素计算input+alpha * other。
- 注意这里有个alpha，叫做乘项因子。类似权重的个东西 这个东西让计算变得更加简洁， 比如线性回归我们知道有个y = wx + b， 在这里直接一行代码torch.add(b, w, x)就搞定。 类似的还有两个方法：

torch.addcdiv(input, value=1, tensor1, tensor2, out=None): 这个实现了

$$\text{out}_i = \text{input}_i + \text{value} \times \frac{\text{tensor1}_i}{\text{tensor2}_i}$$

torch.addcmul(input, value=1, tensor1, tensor2, out=None): 这个实现了

$$\text{out}_i = \text{input}_i + \text{value} \times \text{tensor1}_i \times \text{tensor2}_i$$

这个在优化的时候可能会用到

3. 加法代码示例

```
t_0 = torch.randn((3, 3))
t_1 = torch.ones_like(t_0)
t_add = torch.add(t_0, 10, t_1)

print("t_0:\n{}\nt_1:\n{}\nt_add_10:\n{}".format(t_0,
```

```
## 结果:
t_0:
tensor([[-0.4133,  1.4492, -0.1619],
        [-0.4508,  1.2543,  0.2360],
        [ 1.0054,  1.2767,  0.9953]])
t_1:
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
t_add_10:
tensor([[ 9.5867, 11.4492,  9.8381],
        [ 9.5492, 11.2543, 10.2360],
        [11.0054, 11.2767, 10.9953]])
```

3.2.2 向量运算

向量运算符只在一个特定轴上运算，将一个向量映射到一个标量或者另外一个向量。

```
#单轴计算
a = torch.arange(1,10).float()
print(torch.sum(a))
print(torch.mean(a))
print(torch.max(a))
print(torch.min(a))
print(torch.prod(a)) #累乘
print(torch.std(a))  #标准差
print(torch.var(a))  #方差
print(torch.median(a)) #中位数
```

cum累计计算

```
#cum累计计算
a = torch.arange(1,10)

print(torch.cumsum(a,0))     # 累加
print(torch.cumprod(a,0))     # 累乘
print(torch.cummax(a,0).values)
print(torch.cummax(a,0).indices)
print(torch.cummin(a,0))
```

张量排序

```
#torch.sort和torch.topk可以对张量排序
a = torch.tensor([[9,7,8],[1,3,2],[5,6,4]]).float()
print(torch.sort(a,dim = 1),"\n")
```

• torch.sort(a,dim = 1) 是用来沿着第1维（即列）对每一行进行升序排序。输出是一个元组，第一个元素是一个三行三列的张量，表示每一行排序后的值；第二个元素是一个三行三列的张量，表示每一行排序后值对应的原始列索引。例如，输出中第一行的7、8、9分别是原始张量第一行排序后的值，它们对应的原始列索引分别是1、2、0。

3.2.4 矩阵运算

矩阵必须是二维的。类似torch.tensor([1,2,3])这样的不是矩阵。

矩阵运算包括：矩阵乘法，矩阵转置，矩阵逆，矩阵求迹，矩阵范数，矩阵行列式，矩阵求特征值，矩阵分解等运算。

1. 矩阵乘法

```
#矩阵乘法
a = torch.tensor([[1,2],[3,4]])
b = torch.tensor([[2,0],[0,2]])
print(a@b)  #等价于torch.matmul(a,b) 或 torch.mm(a,b)
```

2. 转置

```
#矩阵转置
a = torch.tensor([[1.0,2],[3,4]])
print(a.t())
```

3. 矩阵求逆

```
#矩阵逆，必须为浮点类型
a = torch.tensor([[1.0,2],[3,4]])
print(torch.inverse(a))
```

4. 矩阵求迹：Trace，表示一个方阵主对角线（从左上角到右下角）的元素之和。

```
#矩阵求trace
a = torch.tensor([[1.0,2],[3,4]])
print(torch.trace(a))
```

5. 求范数和行列式

```
# 矩阵求范数
a = torch.tensor([[1.0, 2], [3, 4]])
print(torch.norm(a))

# 矩阵行列式
a = torch.tensor([[1.0, 2], [3, 4]])
print(torch.det(a))
```

6. 特征值和特征向量

```
# 矩阵特征值和特征向量
a = torch.tensor([[1.0, 2], [-5, 4]], dtype=torch.floa
print(torch.linalg.eig(a))

# 两个特征值分别是 2.5+2.7839j, 2.5-2.7839j
```

7. QR分解

```
#矩阵QR分解，将一个方阵分解为一个正交矩阵q和上三角矩阵r
#QR分解实际上是对矩阵a实施Schmidt正交化得到q

a  = torch.tensor([[1.0,2.0],[3.0,4.0]])
q,r = torch.linalg.qr(a)
print(q,"\n")
print(r,"\n")
print(q@r)
```

8. SVD分解

```
#矩阵svd分解
#svd分解可以将任意一个矩阵分解为一个正交矩阵u,一个对角阵s和一个正
#svd常用于矩阵压缩和降维
a=torch.tensor([[1.0,2.0],[3.0,4.0],[5.0,6.0]])

u,s,v = torch.svd(a)

print(u,"\n")
print(s,"\n")
print(v,"\n")

print(u@torch.diag(s)@v.t())
```

下面基于上面的这些方法搭建一个线性回归模型。

3.1-4. 搭建一个线性回归模型

线性回归是分析一个变量与另外一(多)个变量之间关系的方法。 因变量是y， 自变量是x， 关系线性：

$$y = w \times x + b$$

任务就是求解w，b。

我们的求解步骤：

1. 确定模型： Model -> y = wx + b
2. 选择损失函数： 这里用MSE： $\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$
3. 求解梯度并更新w, b

$$w = w - LR \times w.grad$$
$$b = b - LR \times w.grad$$

这就是我上面说的叫做代码逻辑的一种思路， 写代码往往习惯先有一个这样的一种思路，然后再去写代码的时候，就比较容易了。 而如

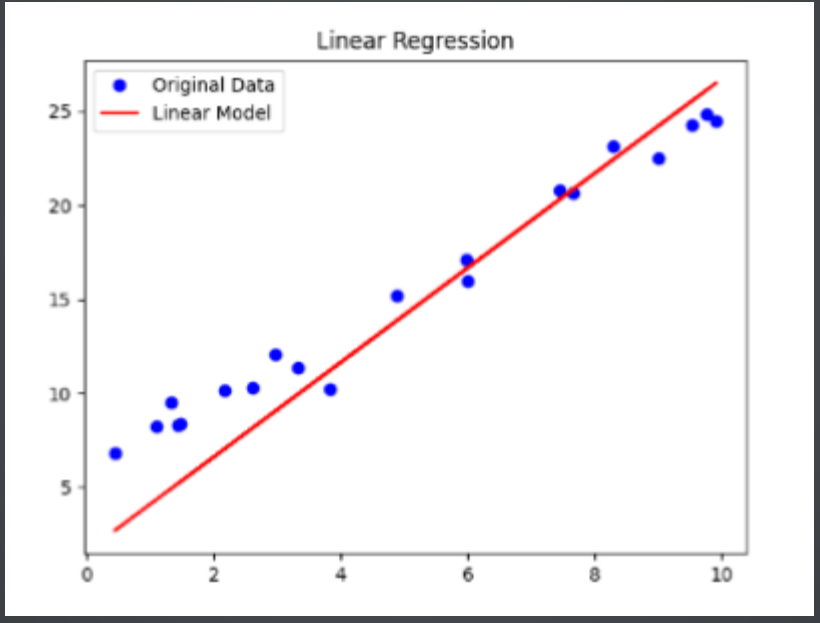
- 如果不系统的学一遍Pytorch， 一上来直接上那种复杂的CNN， LSTM这种，往往这些代码逻辑不好形成
- 因为好多细节我们根本就不知道。 所以这次学习先从最简单的线性回归开始，然后慢慢的到复杂的那种网络。
- 下面我们开始写一个线性回归模型：

```
# 首先我们得有训练样本X，Y， 这里我们随机生成
x = torch.rand(20, 1) * 10
y = 2 * x + (5 + torch.randn(20, 1))

# 构建线性回归函数的参数，随机初始化
w = torch.randn((1), requires_grad=True)
b = torch.zeros((1), requires_grad=True)    # 这俩都需要求

for iteration in range(100):
    # 前向传播
    wx = torch.mul(w, x)
    y_pred = torch.add(wx, b)
```

我们看一下结果：

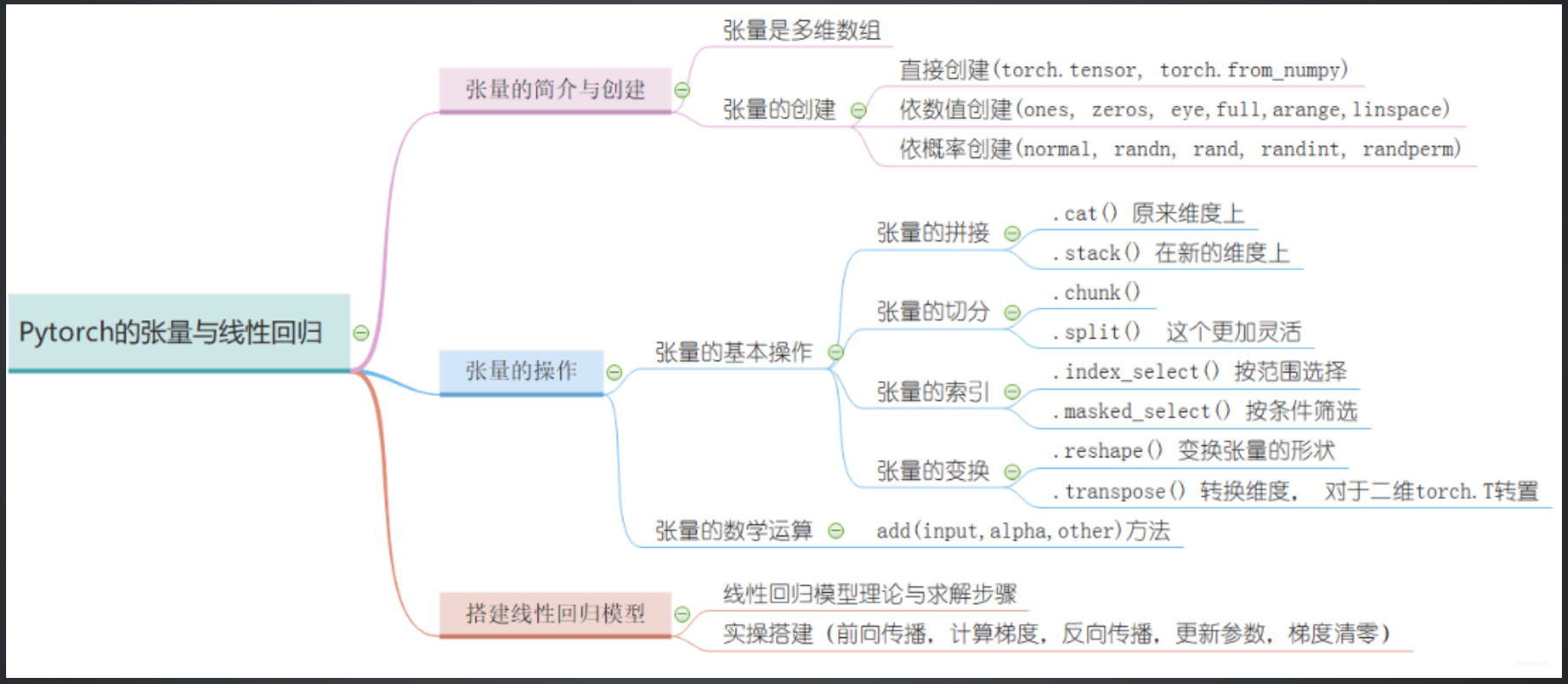

Linear Regression
Original Data
Linear Model
25
20
15
10
5
0
2
4
6
8
10

5. 总结

- 首先我们从Pytorch最基本的数据结构开始，认识了张量什么东西，说白了就是个多维数组
- 然后张量本身有很多的属性， 有关于数据本身的data, dtype, shape, dtype, 也有关于求导的requires_grad, grad, grad_fn, is_leaf。
- 然后学习了张量的创建方法， 比如直接创建，从数组创建，数值创建，按照概率创建等。 这里面涉及到了很多的创建函数tensor(), from_numpy(), ones(), zeros(), eye(), full(), arange(), linspace(), normal(), randn(), rand(), randint(), randperm()等等

- 接着就是张量的操作部分，有基本操作和数学运算
- 基本操作部分
    - 张量的拼接两个函数(.cat, .stack)
    - 张量的切分两个函数(.chunk, .split),
    - 张量的转置(.reshape, .transpose, .t)
    - 张量的索引两个函数(.index_select, .masked_select)
- 数学运算部分，也是很多数学函数，有加减乘除的，指数底数幂函数的，三角函数的……
- 最后基于上面的所学完成了一个简单的线性回归。

- 下面以一张思维导图把这一篇文章的内容概括起来：



- 介绍了很多的函数，每个函数的用法不同
- 具体用法先不用刻意记住
- 先知道哪些函数具体完成什么功能，到时候用的时，边查边用，慢慢的多练才能熟。

下一节学习Pytorch的动态图机制以及自动求导机制， 然后基于前面的这两篇再搭建一个逻辑回归（我们从最简单的模型开始） 😉